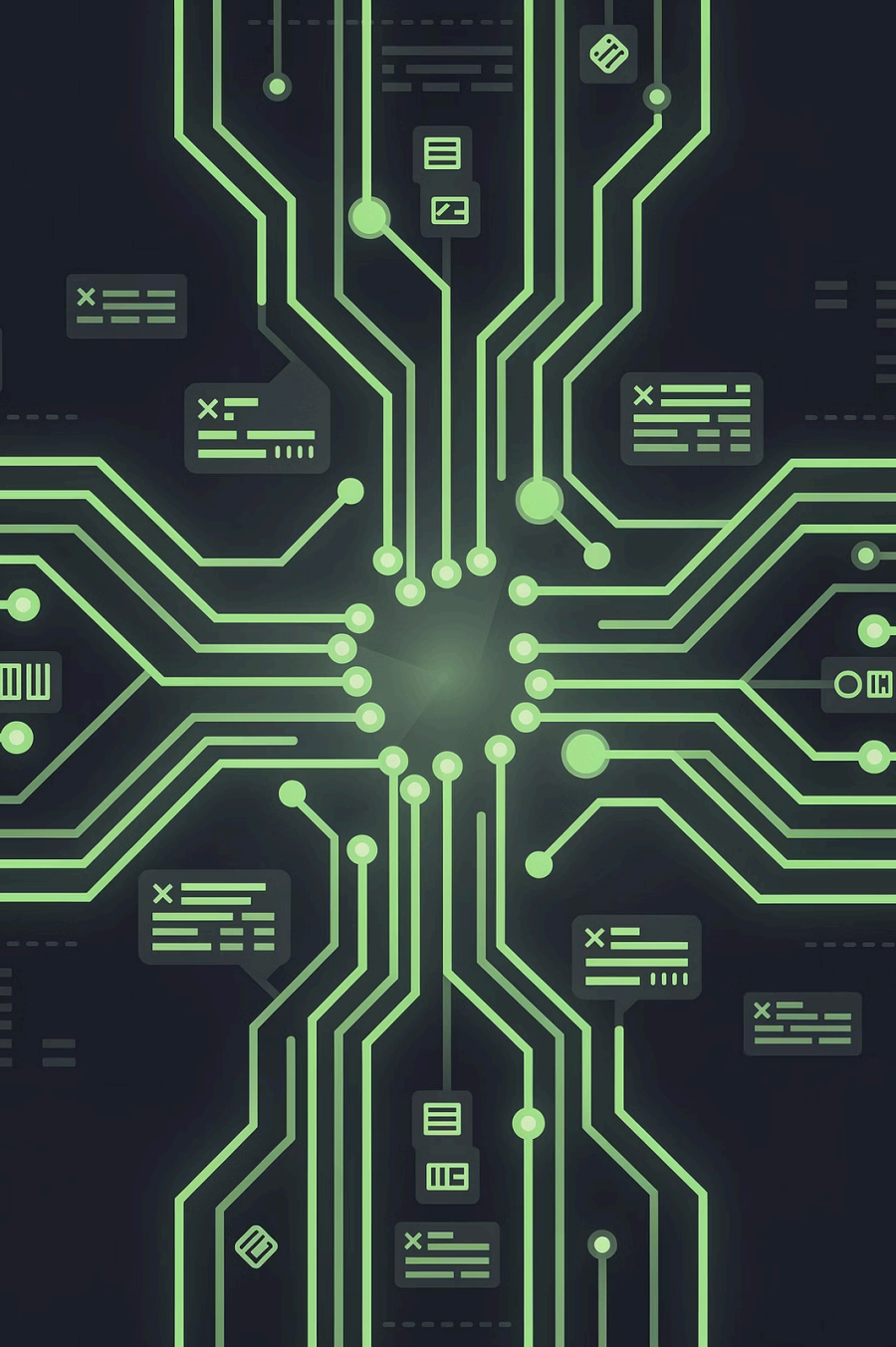




MÓDULO 1

5 HORAS

Introducción a los Lenguajes de Programación

Un recorrido completo por los fundamentos que todo programador necesita dominar: desde la definición de un lenguaje de programación hasta los entornos donde escribirás tu primer código.

MsC Manuel Maldonado — [DGTIC, UNAM](#)

¿Qué vamos a aprender?

Este módulo te llevará paso a paso por los conceptos esenciales de la programación. Cada tema se construye sobre el anterior, formando una base sólida para tu carrera en informática.

01

Lenguaje de programación

Qué es, sus niveles y cómo ha evolucionado a lo largo de la historia.

02

Definición de programa

La relación entre algoritmos y programas: del pensamiento al código.

03

Paradigmas de programación

Las diferentes filosofías para construir software.

04

Traductores | Interpretes

Cómo la computadora entiende lo que escribimos: intérpretes y compiladores.

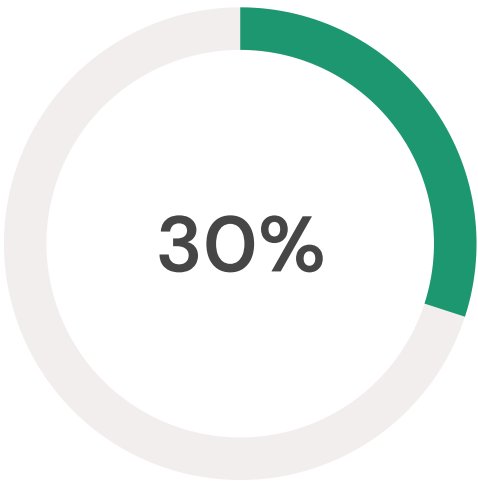
05

Tipos de código e IDEs

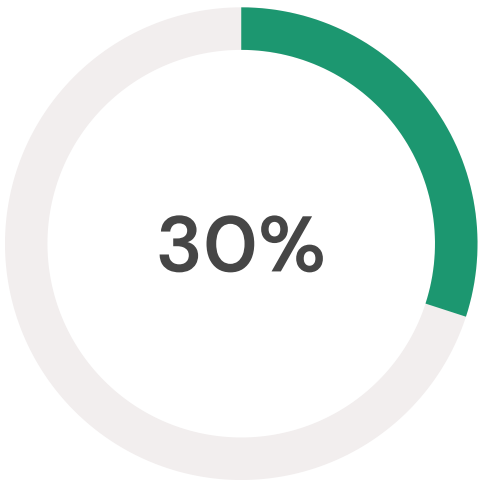
Del código fuente al ejecutable, y las herramientas que facilitan el camino.

Criterios de Calificación

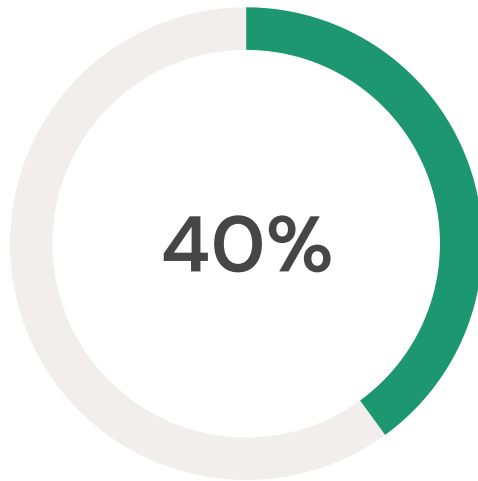
Rubro	Porcentaje	Descripción
Participaciones	30%	Participación activa durante las sesiones de clase, preguntas, discusiones y ejercicios en clase.
Ejercicios Finales	30%	Entrega del archivo .py con los 10 ejercicios de evaluación final correctamente resueltos y comentados.
Prácticas en Clase	40%	Realización y entrega de las prácticas desarrolladas durante las sesiones presenciales.



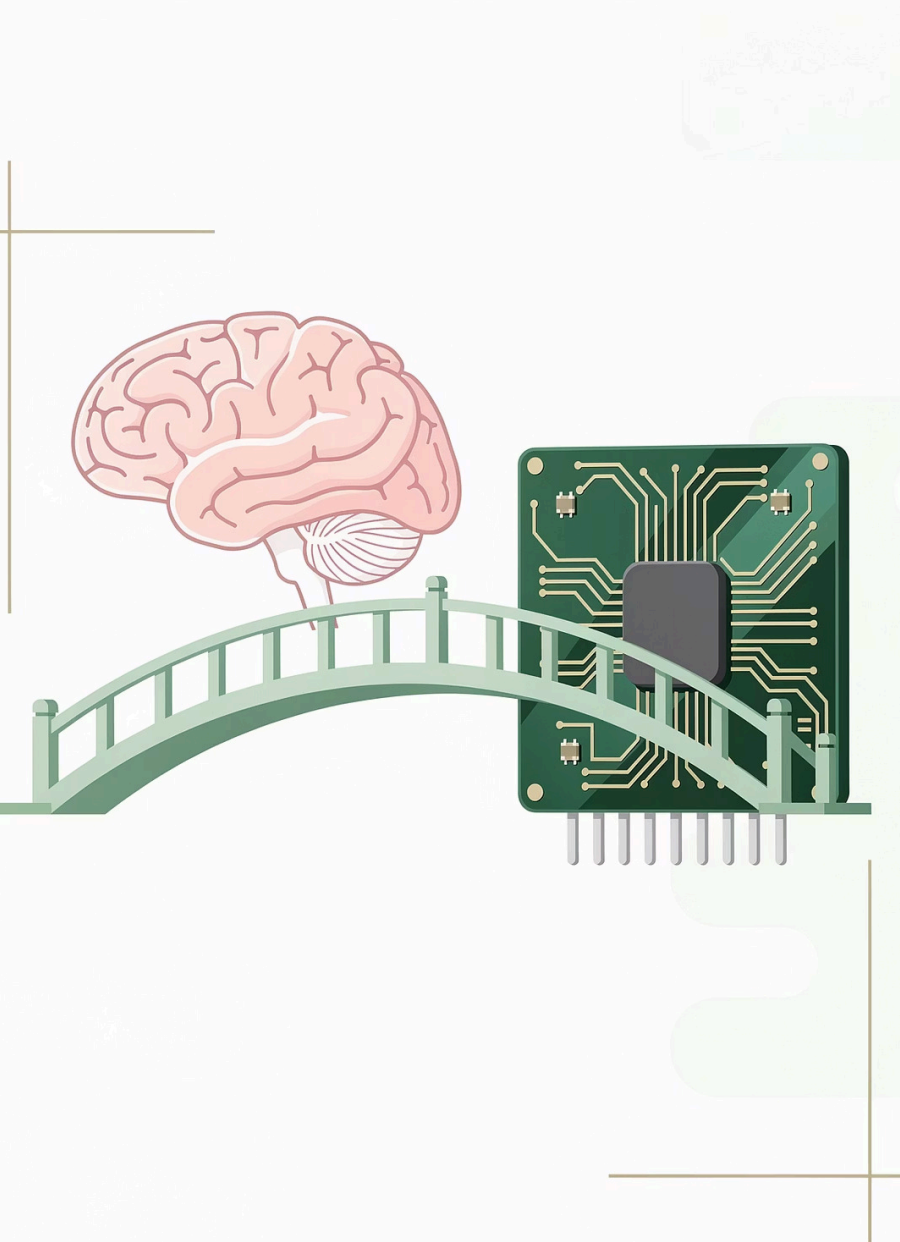
Participaciones



Ejercicios Finales



Prácticas en Clase



TEMA 1.1

¿Qué es un lenguaje de programación?

Un **lenguaje de programación** es un conjunto formal de reglas sintácticas y semánticas que nos permite escribir instrucciones que una computadora puede entender y ejecutar. Funciona como un **punto** entre el pensamiento humano — nuestra lógica, nuestras ideas— y el hardware de la máquina, que solo entiende señales eléctricas representadas como unos y ceros.

Características fundamentales de un lenguaje

Todo lenguaje de programación se define por dos elementos clave que determinan cómo escribimos código y qué significan nuestras instrucciones.

Sintaxis

Son las **reglas** que definen la estructura y combinación correcta de los símbolos. Es similar a la gramática en los idiomas humanos: si no sigues las reglas, la computadora no te entiende.

Por ejemplo, en Python escribimos `print("Hola")` y no `print"Hola"()`.

Semántica

Es el **significado** de las instrucciones escritas correctamente. Aunque la sintaxis sea válida, la semántica determina qué hace realmente el programa.

Por ejemplo, `x = 5 + 3` significa "guarda el resultado de sumar 5 más 3 en la variable x".

Niveles de abstracción

Los lenguajes de programación se clasifican según qué tan cerca están del hardware o del lenguaje humano. A mayor nivel de abstracción, más fácil es programar, pero generalmente se pierde algo de control sobre la máquina.



Bajo nivel

Instrucciones muy cercanas al hardware. Ejemplo: **Ensamblador** usa comandos como `MOV AH, 09h`. Difícil de leer, pero muy eficiente.



Nivel medio

Balance entre control y legibilidad. Ejemplo: **C** permite manipular memoria directamente con `printf("Hola Mundo");`



Alto nivel

Cercanos al lenguaje humano. Ejemplo: **Python** usa `print("Hola Mundo")`. Más fácil de aprender y usar.

"Hola Mundo" en diferentes lenguajes

La mejor forma de ver las diferencias entre niveles es comparar cómo se escribe el mismo programa —un simple saludo— en distintos lenguajes. Observa cómo la complejidad disminuye conforme aumenta el nivel de abstracción.

Lenguaje	Nivel	Paradigma principal	Código para "Hola Mundo"
Ensamblador	Bajo	Imperativo	MOV AH, 09h → INT 21h
C	Medio	Imperativo / Estructurado	printf("Hola Mundo");
Python	Alto	Múltiple	print("Hola Mundo")
Java	Alto	Orientado a objetos	System.out.println("Hola Mundo");

 **Dato curioso:** El programa "Hola Mundo" se popularizó gracias al libro *"The C Programming Language"* de Kernighan y Ritchie en 1978. Desde entonces, es la tradición universal para aprender cualquier lenguaje nuevo.



TEMA 1.2

¿Qué es un programa?

Un **programa informático** es una secuencia de instrucciones escritas en uno o más lenguajes de programación que, cuando son ejecutadas por una computadora, producen un resultado específico. Es, en esencia, la **implementación concreta** de un algoritmo: el momento donde una idea abstracta cobra vida dentro de la máquina.

Del algoritmo al programa

Antes de escribir código, necesitamos un **algoritmo**: un conjunto de pasos ordenados y finitos para resolver un problema. El algoritmo es independiente del lenguaje; el programa es su traducción a un lenguaje específico.

Algoritmo (Pseudocódigo)

Problema: Calcular el área de un rectángulo

1. INICIO
2. LEER base, altura
3. $\text{area} = \text{base} \times \text{altura}$
4. MOSTRAR area
5. FIN

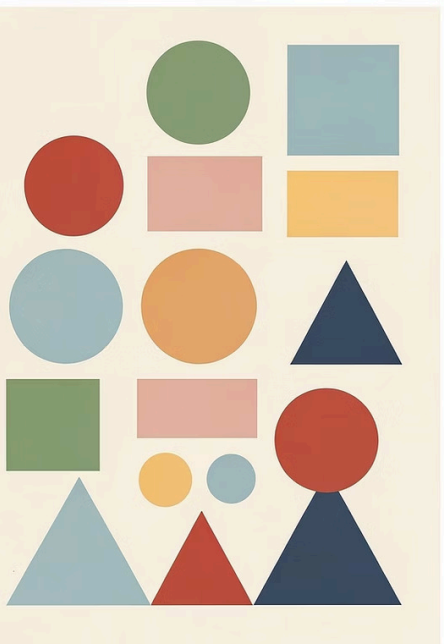
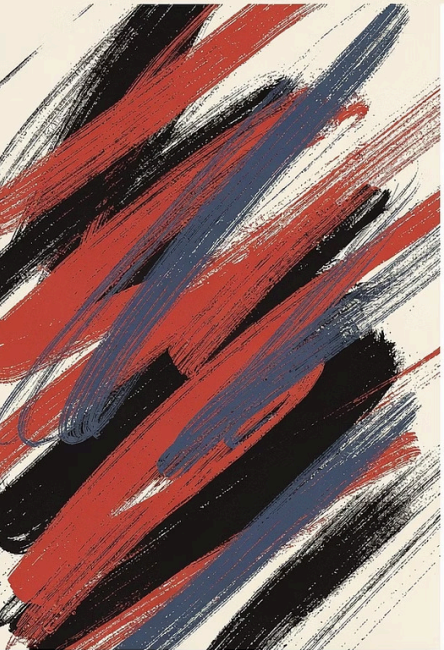
Programa en Python

```
base = 5
altura = 3
area = base * altura
print(area) # Resultado: 15
```

Programa en C

```
#include <stdio.h>
int main() {
    int base = 5, altura = 3;
    int area = base * altura;
    printf("%d", area);
    return 0;
}
```

Observa cómo el **mismo algoritmo** se puede implementar en lenguajes distintos. La lógica es idéntica; solo cambia la sintaxis.



TEMA 1.3

Paradigmas de la programación

Un **paradigma de programación** es un estilo o enfoque fundamental para construir programas. Así como en el arte existen corrientes (realismo, impresionismo, abstracto), en la programación existen diferentes formas de pensar y organizar el código. Cada paradigma tiene sus propias reglas, ventajas y situaciones ideales de uso.

Los 4 paradigmas principales

Conocer los paradigmas te permitirá elegir la herramienta adecuada para cada problema. Muchos lenguajes modernos como Python son **multiparadigma**, es decir, soportan varios enfoques a la vez.

Imperativo / Estructurado

Instrucciones paso a paso con bloques de control: secuencia, selección e iteración. Evolución que eliminó el uso de goto.

Lenguajes: C, Pascal

Funcional

Basado en funciones matemáticas puras. Evita cambios de estado y datos mutables. Promueve código predecible y conciso.

Lenguajes: Haskell, Lisp

Orientado a objetos

Organiza el código en "objetos" con atributos y métodos. Principios: encapsulamiento, herencia y polimorfismo.

Lenguajes: Java, C#

Lógico

Define hechos y reglas; el sistema usa inferencia lógica para encontrar respuestas. El programador describe *qué*, no *cómo*.

Lenguajes: Prolog

Paradigmas en acción: comparemos el código

Veamos cómo se resuelve un mismo problema —**calcular el factorial de 5**— usando diferentes paradigmas. Esto te ayudará a entender que no hay una sola forma "correcta" de programar.

Estructurado (C)

```
int factorial(int n) {  
    int resultado = 1;  
    for(int i = 1; i <= n; i++) {  
        resultado *= i;  
    }  
    return resultado;  
}
```

Usa un ciclo `for` para multiplicar paso a paso. Es un enfoque **imperativo**: le dices a la máquina *cómo* hacerlo.

Funcional (Haskell)

```
factorial 0 = 1  
factorial n = n * factorial (n-1)
```

Usa **recursión pura** y definición matemática. Le dices a la máquina *qué es* un factorial, no cómo calcularlo.

Orientado a objetos (Java)

```
public class Calculadora {  
    public int factorial(int n) {  
        if (n == 0) return 1;  
        return n * factorial(n-1);  
    }  
}
```

El método vive dentro de un **objeto** que encapsula la funcionalidad.

Programación lógica: un paradigma diferente

La programación lógica es quizá el paradigma más distinto a lo que imaginas como "programar". En lugar de dar instrucciones, **declaras hechos y reglas**, y luego haces preguntas al sistema. El motor de inferencia se encarga de encontrar las respuestas.

Ejemplo en Prolog

```
% Hechos
progenitor(pedro, juan).
progenitor(juan, ana).

% Regla
abuelo(X, Y) :-
    progenitor(X, Z),
    progenitor(Z, Y).

% Consulta
?- abuelo(pedro, ana).
% Respuesta: true
```

¿Cómo funciona?

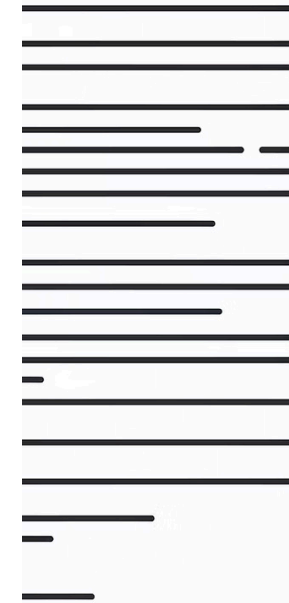
- **Hechos:** Pedro es progenitor de Juan; Juan es progenitor de Ana.
- **Regla:** X es abuelo de Y si X es progenitor de Z y Z es progenitor de Y.
- **Consulta:** ¿Pedro es abuelo de Ana? → El sistema infiere que sí.

Tú defines *la lógica*; Prolog encuentra *la respuesta*.

Traductores

Las computadoras solo entienden **código máquina** (unos y ceros). Entonces, ¿cómo logran ejecutar el código que escribimos en Python o Java? La respuesta está en los **traductores**: programas especializados que convierten nuestro código a un lenguaje que la máquina puede procesar. Existen dos tipos principales: **intérpretes** y **compiladores**.

Translation



0001110110
0101111010
1101010110
0010010010
0100110110
0011010010
1111010110
0011110110
1011110110
1010110101
0010010110
1011010010
0101011010
1101010010
0001110000
1011010100
1010110110
1000000010
0000000000



Intérpretes vs. Compiladores

Entender la diferencia entre estos dos tipos de traductores es fundamental para comprender cómo funcionan los lenguajes que usarás a lo largo de tu carrera.

Intérprete

Traduce y ejecuta el programa **línea por línea**, en tiempo real. Si encuentra un error, se detiene inmediatamente en esa línea.

✓ **Ventajas:** Mayor portabilidad, fácil depuración, ideal para aprender.

✗ **Desventajas:** Menor velocidad de ejecución.

Ejemplos: Python, JavaScript, Ruby

Compilador

Traduce **todo el programa** a código máquina antes de ejecutarlo. Genera un archivo ejecutable independiente.

✓ **Ventajas:** Mayor velocidad de ejecución, optimización del código.

✗ **Desventajas:** Menor portabilidad, más pasos para probar cambios.

Ejemplos: C, C++, Go, Rust

💡 🤔 **¿Sabías que Java es híbrido?** Se compila a un formato intermedio llamado *bytecode*, que luego es interpretado por la Máquina Virtual de Java (JVM). Esto le da portabilidad y buen rendimiento.

Comparación detallada de traductores

Aspecto	Intérprete	Compilador
Proceso	Lee y ejecuta línea a línea	Lee todo, traduce y luego ejecuta
Velocidad	Más lenta en ejecución	Más rápida en ejecución
Errores	Se detiene en el primer error	Muestra todos los errores al compilar
Resultado	No genera archivo independiente	Genera un ejecutable (.exe, .out)
Portabilidad	Alta (solo necesita el intérprete)	Baja (depende de la plataforma)



Python — Interpretado

Se compila internamente a bytecode, pero el usuario lo experimenta como interpretado.



C — Compilado

Se compila directamente a código máquina nativo de la plataforma.

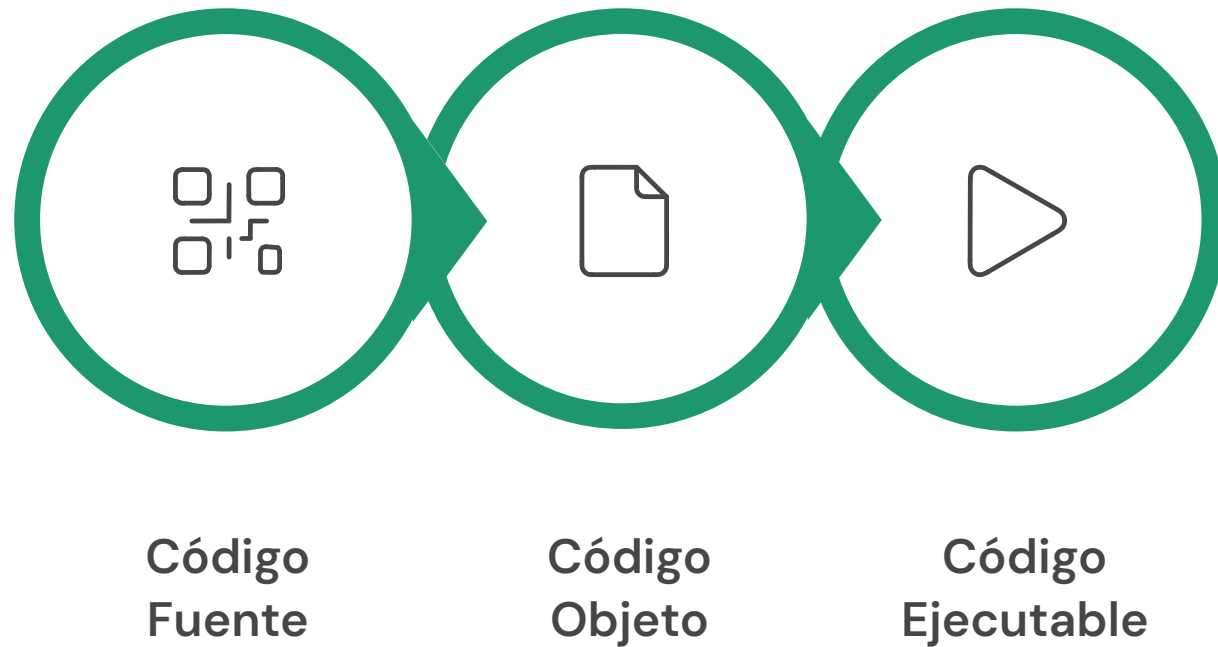


Java — Híbrido

Se compila a bytecode que es interpretado por la JVM, combinando lo mejor de ambos mundos.

Tipos de código

Durante el desarrollo y ejecución de un programa, el código atraviesa diferentes estados o transformaciones. Entender estas etapas es clave para comprender el ciclo de vida completo del software.



El **código fuente** es lo que escribes como programador, legible por humanos. El **código objeto** es el resultado intermedio de la compilación: binario pero incompleto. Finalmente, el **código ejecutable** es el archivo que el sistema operativo puede correr directamente.

El proceso completo en C: de fuente a ejecutable

Veamos un ejemplo concreto del recorrido que hace tu código desde que lo escribes hasta que se ejecuta en la computadora.

Código fuente (programa.c)

1

```
#include <stdio.h>
int main() {
    printf("Hola Mundo\n");
    return 0;
}
```

📁 Extensiones comunes: .c, .py, .java, .js

Código objeto (programa.o)

Contenido binario no legible por humanos. Si usas `hexdump` verías algo como:

2

```
7f 45 4c 46 02 01 01 00 ...
```

📁 Extensiones: .obj (Windows), .o (Linux/Unix)

Código ejecutable (programa.exe)

3

Archivo binario completo, listo para ser cargado en memoria y ejecutado por el sistema operativo.

📁 Extensiones: .exe (Windows), sin extensión (Linux), .app (macOS)

Entornos de Desarrollo Integrado (IDE)

Un **Entorno de Desarrollo Integrado** (IDE, por sus siglas en inglés) es una aplicación que reúne todas las herramientas necesarias para desarrollar software **en un solo lugar**. Su objetivo es maximizar tu productividad reduciendo el tiempo de configuración y el cambio de contexto entre herramientas separadas.



Editor de código

Resaltado de sintaxis, autocompletado inteligente y detección de errores en tiempo real.



Depurador

Ejecuta código paso a paso, establece puntos de interrupción e inspecciona variables.



Compilador / Intérprete

Compila o ejecuta tu código directamente con un solo clic.



Terminal integrada

Acceso a línea de comandos sin salir del entorno de trabajo.

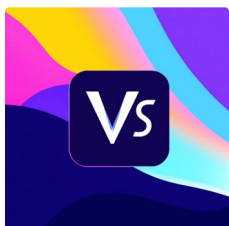


Control de versiones

Integración con Git para hacer commit, push y pull desde la interfaz visual.

IDEs populares: ¿cuál elegir?

Existen muchas opciones de IDEs, tanto de escritorio como en la nube. Aquí te presentamos los más utilizados en la industria y en el ámbito académico.



VS Code

Lenguajes: Múltiples (Python, JS, C++, etc.)

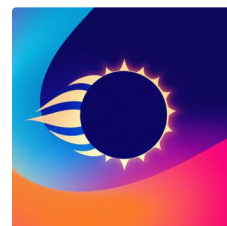
Ligero, extensible, gratuito.
Enorme ecosistema de extensiones. **El más popular actualmente.**



IntelliJ IDEA

Lenguajes: Java, Kotlin, Scala

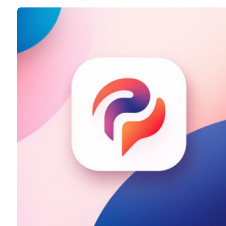
Potente para desarrollo JVM.
Versión Community gratuita disponible.



Eclipse

Lenguajes: Java, C/C++, PHP

Multipropósito, gratuito, gran comunidad y amplio soporte de plugins.



PyCharm

Lenguajes: Python

Especializado en Python con versiones Community (gratuita) y Professional.

☞ ☁ **IDEs en la nube:** Herramientas como **GitHub Codespaces**, **Replit** y **AWS Cloud9** permiten programar directamente desde el navegador, sin instalar nada en tu equipo. ¡Ideales para empezar a practicar hoy mismo!

Lo que aprendimos hoy

Hemos recorrido los **6 pilares fundamentales** que sustentan todo el mundo de la programación. Con estos conceptos claros, estás listo para empezar a escribir tus primeros programas.

1. Lenguaje

Sintaxis + Semántica. Bajo y alto nivel.

2. Programa

Algoritmo traducido a código ejecutable.

3. Paradigmas

Estructurado, funcional, POO y lógico.

4. Traductores

Intérpretes, compiladores e híbridos.

5. Tipos de código

Fuente → Objeto → Ejecutable.

6. IDEs

Tu taller completo: VS Code, Eclipse, PyCharm.

Actividades para reforzar

Discusión grupal

¿Qué lenguajes conocen y a qué paradigma creen que pertenecen?

Ejercicio de identificación

Identifica el paradigma en fragmentos de código reales.

Práctica con IDE

Abre VS Code, crea un archivo y ejecuta tu primer "Hola Mundo".

Operadores de Comparación y Lógicos

Los operadores de comparación y lógicos son fundamentales para controlar el flujo de un programa. Permiten evaluar condiciones y combinarlas para tomar decisiones.

Operadores de Comparación (Condicionales)

Operador	Significado	Ejemplo (en C/Java/JavaScript)	Resultado del ejemplo
==	Igual a	5 == 5 / 5 == 3	true / false
!=	Diferente de	5 != 3 / 5 != 5	true / false
<	Menor que	3 < 5 / 5 < 3	true / false
>	Mayor que	5 > 3 / 3 > 5	true / false
<=	Menor o igual que	3 <= 5 / 5 <= 5 / 6 <= 5	true / true / false
>=	Mayor o igual que	5 >= 3 / 5 >= 5 / 4 >= 5	true / true / false

Operadores Lógicos (Combinan condiciones)

Operador	Significado	Ejemplo (en C/Java/JavaScript)	Resultado
&&	Y (AND)	(5 > 3) && (2 < 4) / (5 > 3) && (2 > 4)	true / false
	O (OR)	(5 > 3) (2 > 4) / (5 < 3) (2 > 4)	true / false
not	NO (NOT)	!(5 > 3) / !(5 < 3)	false / true

Notas importantes según el lenguaje

Igualdad estricta vs. débil

JavaScript: == compara solo valor (con conversión de tipo), === compara valor y tipo (estricta). Igual para != y !==.

Python: == y != comparan valor y tipo. Ejemplo: 5 == 5.0 → True, pero 5 == "5" → False.

Java y C#: Para primitivos, == compara valor. Para objetos, == compara referencias; usar .equals() para contenido.

Cortocircuito en operadores lógicos

En casi todos los lenguajes, && y || evalúan la segunda expresión solo si es necesario. Útil para evitar errores.

Ejemplo: if (divisor != 0 && (dividendo / divisor) > 10)

Operadores lógicos en Python

En Python se usan palabras clave en lugar de símbolos: and, or, not en lugar de &&, ||, !. Ejemplo: if (x > 5 and y < 10):